

```

#ifndef GROUP_H
#define GROUP_H

#include "linked_list.h"
#include "gen_table.h"
#include <assert.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

typedef struct group {
    char *name; // Nom
    int n; // Cardinal du groupe
    int **table; /* Table de Cayley:
                  le groupe est assimilé
                  à [0, 1, ..., n-1]
                  */
} group_t;

typedef struct group_data {
    gentabl_t* rel; // Table des décompositions
    list_t* gen; // Liste des générateurs
} gdata_t;

typedef struct decomp_abelian_s {
    int n;
    int num_basis;
    int* basis;
    int** basis_decomps;
} decomp_t;

void add_sum_to_decomp(int x, int y, decomp_t* d);

group_t *group_empty(int n);
void group_name(group_t *g, char *s);
void group_free(group_t *g);
void group_print(group_t *g);

gdata_t* create_empty_gdata();
gdata_t* parse_entries(list_t* gen, gentabl_t* rel);
void free_gdata(gdata_t* gdata);

list_t* get_generator(gdata_t* gdata);
gentabl_t* get_rel(gdata_t* gdata);

group_t *from_matrix(int **matrix);
group_t *copy_group(group_t *g);

int calculate(group_t* g, int x, int y);
bool is_neutral(group_t* g, int x);
int get_neutral(group_t* g);
int calculate_order(group_t* g, int x);
int* get_orders_list(group_t* g);

int calc_generated_subgroup(group_t* g, int gen, list_t* generators, gentabl_t* decomp);
gdata_t* calc_gen_and_rel(group_t* g);
list_t* get_cyclic_subgroup_element(group_t* g, int x, int e);
bool are_isomorphic(group_t* g1, group_t* g2);
bool are_isomorphic_with_order(group_t* g1, group_t* g2);

void print_array(int* arr, int n);

bool are_isomorphic_with_order_custom_gen(group_t* g1, group_t* g2, int* generators, int
num_gen);

```

```
bool are_isomorphic_probabilistic(group_t* g1, group_t* g2, int max_attempts);
bool are_isomorphic_hybrid(group_t* g1, group_t* g2, long long deterministic_threshold, int
probabilistic_attempts);
bool are_isomorphic_adaptive(group_t* g1, group_t* g2);
#endif
```